ServeNet-LT: A Normalized Multi-head Deep Neural Network for Long-tailed Web Services Classification

Jing Zhang†, Yang Chen†, Yilong Yang†‡*, Changran Lei†, Deqiang Wang†
†School of Software, Beihang University, Beijing, China
‡State Key Laboratory of Software Development Environment, Beijing, China
Email: zhang_jing@buaa.edu.cn

***Abstract*—Automatic service classification plays an important role in service discovery, selection, and composition. Recently, machine learning has been widely used in service classification. Though promising results are obtained, previous methods are merely evaluated on web services datasets with small-scale data and relatively balanced data, which limit their real-world applications. In this paper, we address the long-tailed web services classification problem with more categories and imbalanced data. Due to the long-tailed distribution of datasets, the existing machine learning and deep learning methods cannot work well. To deal with the long-tailed problem, we propose a normalized multi-head classifier learning strategy, which effectively reduces the classifier bias and benefit the generalization capacity of the extracted features. Extensive experiments are conducted on a large-scale long-tailed web services dataset, and the results show that our model outperforms the 11 compared service classification methods to a large margin.**

***Index Terms*—Deep Learning; Service; Web Services; Service Classification; Long-tailed Distributions**

I. Introduction

In recent decades, software reuse is an effective way to reduce the cost of software development. Web services provide the possibility of software reuse based on independent, low coupling, self-contained, programmable web-based architecture [1]. These web services repositories allow service providers to publish services using uniform specifications, including service descriptions, service names, URL, and search keywords (labels or tags) in natural language. Two popular web services repositories are the standard Universal Description, Discovery, and Integration (UDDI) registry [2] and APIs sharing platform. The key of software reuse is to find the required services in the web services repository according to the service specification, which is the primary problem of service discovery [3].

The most common methods of service discovery are keyword search and semantic search [4] [5]. Keyword search requires the service consumer to search the target service through the key, and semantic search requires matching the target service according to the service signature. Although semantic search methods have higher accuracy than keyword-based methods, they have strict restrictions in use, which makes the efficiency of semantic search methods lower than keyword-based methods. In practice, the preferred method for most tasks is based on keywords. The success of keyword-based search largely depends on the quality of service keywords manually assigned by developers. However, the specified keywords are not always reliable and sufficient. This is mainly because developers may find it difficult to select the best keywords from a large database of candidates and lack the knowledge of all candidates. Due to the limitation of manual keyword assignment, it is necessary to use machine learning methods for automatic keyword prediction and tag recommendation.

In practice, web services are gradually categorized into more fine-grained categories as the increasingly growing requirements of the precise search and the appearance of new types of services. The newly appeared services with fewer samples may potentially involve more advanced techniques. Thus, not only the commonly used categories (i.e. head categories) with more service samples are required to be classified accurately, but categories with fewer samples (i.e. tail categories) are also of high importance. It is challenging for machine learning and deep learning methods to classify accurately the categories due to the long-tailed distribution of data. Specifically, under the long-tailed data, the classification results are severely biased towards the data-rich head categories. This is why other machine methods cannot be directly used.

At present, in the field of services, there is very little work to eliminate the impact of long-tailed distribution. In the machine learning community, the most commonly used methods for dealing with the long-tailed distributed data include instance re-sampling [6] [7] [8] or re-weighting [9] [10] [11] [12]. For example, the instance re-sampling method samples the data in the head categories and duplicates the data in the tail categories to re-balance the data. Similarly, in instance re-weighting, the losses of different samples are re-weighted based on the proportion of the categories too, i.e. The head (tail) categories are assigned with smaller (larger) weights. However, in practice, these methods may suffer from more explanations on the head categories. It is very challenging but meaningful to effectively deal with the impact of the long-tailed distribution of data.

In this paper, we propose a novel end-to-end deep neural network designed for long-tailed service classification with

*Corresponding author: Yilong Yang (yilongyang@buaa.edu.cn)

2021 IEEE International Conference on Web Services (ICWS) | 978-1-6654-1681-8/21/$31.00 ©2021 IEEE | DOI: 10.1109/ICWS53863.2021.00025

Authorized licensed use limited to: Penn State University. Downloaded on June 17,2025 at 14:19:01 UTC from IEEE Xplore. Restrictions apply.

more service categories. To deal with the long-tailed classification, we propose a multi-head classifier learning strategy, which effectively reduces the classifier bias and benefit the generalization capacity of the learned features. The motivation is that the machine learning methods, especially the deep learning models, generally find the shortcut connection between the input data and the output predictions. In the context of long-tailed classification, the classifier tends to be converged when it yields correct predictions for the majority of training examples (i.e. the head categories). Thus, the learned representations are normally biased towards the head categories regardless of the tail categories. To reduce the biases, the proposed normalized multi-head classifier forces the deep learning models to learn more diverse features rather than the most representative ones, and thus the biases are alleviated to a large degree. Moreover, the mechanism of features and classifier weights normalization can further reduce the biases. The experimental results verify that the proposed method can effectively improve the classification results of the tail categories to a large margin without many detrimental effects on the head categories.

The contributions of this paper are summarized as follows:

- Based on the real-world requirements, we identify and address a more challenging but practical problem in web services classification, where more categories are involved and the long-tailed issue of data is considered.
- To deal with the large-scale and long-tailed service classification problem, we propose a new normalization multi-head classification mechanism into an end-to-end deep learning framework.
- Extensive experiments conducted on a large-scale Web services dataset that includes up to 300 categories verified the effectiveness of the proposed method.

The remainder of the paper is organized as follows: Section 2 reviews the related work and the details of comparison between the original ServeNet [13] and the new proposed *ServeNet-LT*. Section 3 presents the proposed *ServeNet-LT*. Section 4 conducts the experimental evaluation on the comparison between the proposed *ServeNet-LT* and other web services classification methods. Section 5 concludes the paper and identifies the future work.

II. Related work

This paper focuses on web services classification by dealing with the issue of the long-tailed distribution of data. In the web services classification task, we review the traditional machine learning method, deep learning method, and the latest service classification method. For the issue of long-tailed distribution of data, we review the latest long-tailed classification methods of deep learning and data processing methods.

A. Web Services Classification Methods

Conventional machine learning methods have been widely used to predict the tags of service. In the work [14], Fang et al. use Web Services Description Language (WSDL) documents to cluster services and predict the tags according to the tags of other services in the same cluster. The work in [15] [16] compares several machine learning methods such as Naive Bayes [17], Support Vector Machines (SVM) [18], k-Nearest Neighbors (kNN) [19], and C4.5 [20] for service classification on 7 categories, the result demonstrates that SVM has the best accuracy among all machine learning methods. Some studies try to combine methods to obtain higher accuracy. Kapitsaki et al. [21] adopt ensemble schemes by combining the classifiers among Naive Bayes, KNN, and C4.5 based on service description and WSDL. By integrating Latent Dirichlet Allocation (LDA) [22] with SVM for feature extractions, Liu et al. [23] show that the LDA-SVM model can reach around 90% accuracy on 10 categories, but only less than 50% accuracy can be achieved when predicting multi-label data [24] [25]. The effect of the methods based on machine learning depends on the quality of feature engineering, which is hard and expensive [26].

Deep learning takes a step further and has no requirement for feature engineering [27]. It has been successfully applied to many fields. In image classification, Convolutional Neural Network (CNN) [28] can achieve less than 5% Top-5 errors of 1000 categories on the ImageNet dataset [29]. For text classification, the Recurrent Neural Networks (RNN) and Long Short-Term Memory (LSTM) has shown strong and effective [30], and CNN has also been applied to solve text classification problems [31]. Recently, several studies have applied deep learning to the field of web services discovery. The work [32] presents a deep neural network for web services classification, named ServeNet, to automatically extract features from service descriptions by adopting BERT pre-trained model. In work [13], they improve ServeNet and reach 91.58% Top-5 accuracy and 69.95% Top-1 accuracy on 50 categories by abstracting additional features from service name to deal with the issue of input length limitations. The work [33] present an integrated framework for semantic service composition. DeepWSC [34] proposes a framework to cluster services based on Recurrent-CNN. The work [35] [36] design a web services classification framework based on GCN. By extracting features from service description documents and co-invocation information of mashup APIs respectively, dual-GCN can suppress noise propagation of textual contents and classify web services. The work [37] proposes an unsupervised Bayesian probabilistic model bi-HPTM to reach 66.9% Top-1 accuracy on 50 categories and 72.9% on 20 categories by capturing the textual semantics of service descriptions.

In short, all the above methods are limited in the number of service categories, which limit their applications in real-world scenarios. Moreover, they do not take the long-tailed distribution of data into account and thus the performance of their methods is sub-optimal. We propose a novel method to expand the number of categories to 300 categories and explicitly deal with the long-tailed issue of data.

B. Long-tailed Classification Methods

In long-tailed classification, the categories with more (less) number of samples are called the head (tail) categories.

Authorized licensed use limited to: Penn State University. Downloaded on June 17,2025 at 14:19:01 UTC from IEEE Xplore. Restrictions apply.

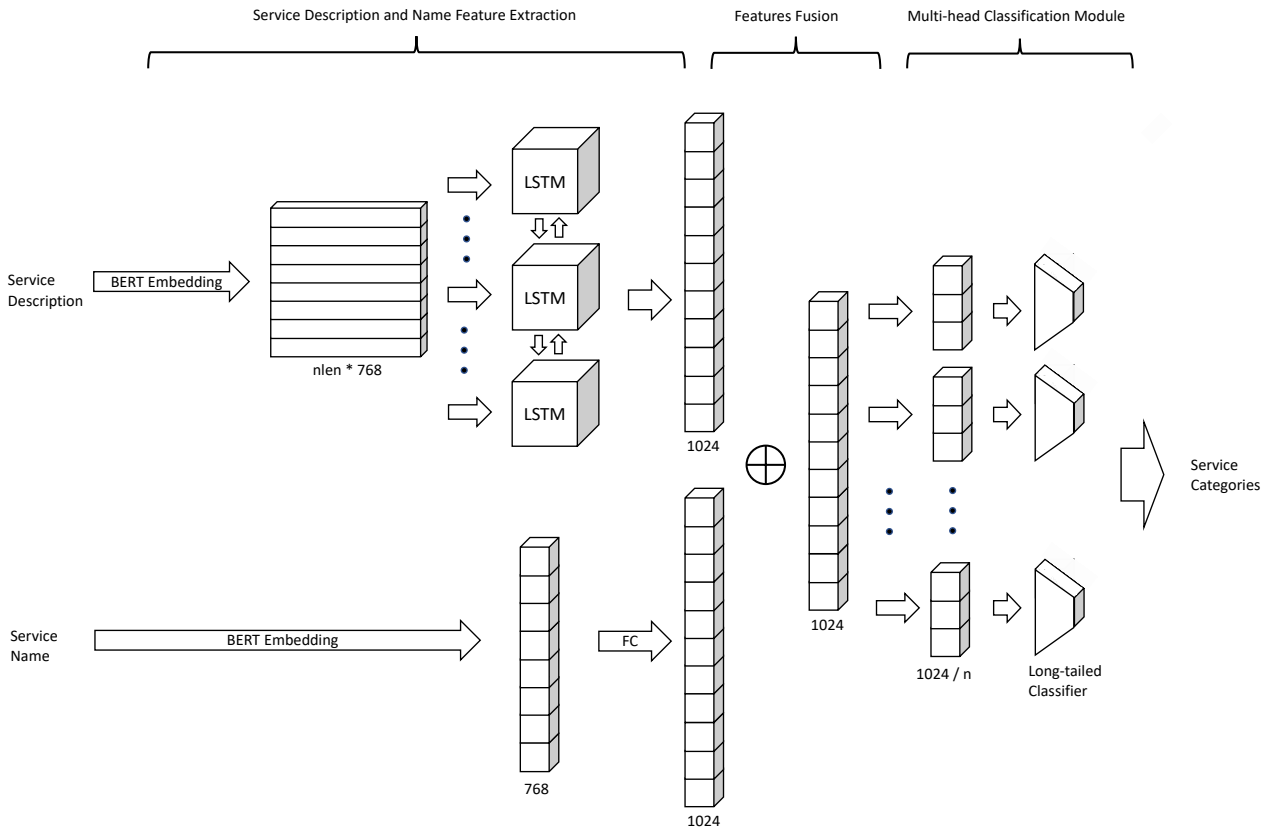

Fig. 1: *ServeNet-LT* architecture: The modal contains three main parts: 1) service description and name feature extraction through the BERT model, 2) the feature fusion via the trainable weighted element-wise sum, 3) the proposed multi-head classification module. Note that the classification module is distinguished from conventional classification layers from two main aspects: a) the features are segmented into multiple segments, b) an individual classifier is applied on each part and the classifiers are normalized.

Some methods achieve the balanced data distribution by re-sampling, including over-sampling [6] [7] for the tail category by duplicating data, and under-sampling [8] for the data-rich category by removing data. Similarly, the idea of re-weighted [9] [10] [11] [12]. Training allows the scale of the losses to be changed at the category level by assigning different weights to different training categories, such that the tail categories are emphasized during training. Focal loss [38] is a state-of-the-art re-weighting method, which proposes a new loss function to alleviate data imbalance. The loss function can reduce the weights of samples in the head categories and emphasize more on the tail categories. Different balancing strategies have different effects on the long-tailed distribution. The two-stage training [39] decouples the learning procedure into representation learning and classification. De-confounded training [40] proposes a causal perspective and treat the momentum in the optimizer of a deep learning model as the cause of the bias towards the head classes. They propose to calculate the moving average vector of the feature to calibrate the distribution of features to solve the long-tail problem. The DCL [41] method adaptively adjusts the sampling strategy and loss weight in each batch. The Attention-based [42] method is also used for the long-tailed problems. For example, LMA [43] integrates label embedding into attention module for web services classification tasks with a transformer-like structure, and the OLTR [44] combines a direct image feature and an associated memory feature through dynamic meta-embedding. In addition, some works [45] [46] deal with long-tailed recognition problem with class-balanced methods. In short, the long-tailed distribution of data brings great challenges to deep learning. To the best of our knowledge, in the field of web services classification, there is

Authorized licensed use limited to: Penn State University. Downloaded on June 17,2025 at 14:19:01 UTC from IEEE Xplore. Restrictions apply.

little research on long-tailed distribution. Moreover, most of the current long-tailed classification methods may improve the tail-class performances at the cost of degradation of the head-class. We propose a novel normalized multi-head classification method to obtain the state-of-the-art performance on the long-tailed benchmark dataset.

III. SERVENET-LT

In this section, we introduce the details of the proposed Long-tailed ServeNet, named *ServeNet-LT*. Suppose the web service dataset is $\mathcal{D} = \{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i=1}^{N}$ with N labeled training data, where $\boldsymbol{x}_i$ denotes the input web service data (e.g. service name and service description) while $\boldsymbol{y}_i$ indicates the corresponding class label of $\boldsymbol{x}_i$. We denote the number of classes as K, then we encode the label of a web service as a one-hot vector as $\boldsymbol{y}_i = [y_i^1, ..., y_i^K] \in \{0, 1\}^K$. The goal of our *ServeNet-LT* model is to correctly classify the web services based on the training data.

A. The ServeNet-LT Method

The proposed deep neural network *ServeNet-LT* consists of four modules: a) service description feature extraction module, b) service name feature extraction module, c) feature fusion module, and d) multi-head classification module. The structure overview of *ServeNet-LT* is shown in Fig. 1.

1) Service Description Feature Extraction: The backbone used in this paper is slightly different from the previous ServeNet [13]. We remove the convolution layer, which is helpful for local feature extraction but brings little performance gain for web services classification problems. Therefore, we remove the convolution layer to ensure accuracy and make the model more efficient by reducing the number of trainable parameters. In order to show the effectiveness of our long-tailed method, we use the same strategy as the previous ServeNet [13] in data embedding and mini-batch processing. Specifically, we use the BERT pre-trained language model for embedding the text data, which can embed each word to an n-dimensional vector. The BERT pre-trained language model has two types of embedding output: f_{bert_pool} embeds a whole sentence into a single vector, while f_{bert_seq} embeds each word of a sentence to an individual vector. In particular, we embed every word in the service description into a vector through BERT pre-trained language model, which is represented by f_{bert_seq}. After embedding, service description outputs a $nLen$ by n description matrix, which is represented by E. We assume that x_1 is the description of a service. The embedding of service description f_{bert_seq} can be defined as:

$$E = f_{bert_seq}(x_1) \tag{1}$$

Different from the previous ServeNet [13], for feature extraction, we adopt a Bi-LSTM model to further enhance the position information in service descriptions and obtain the global features. We experimentally verified that a simple Bi-LSTM can achieve comparable results because it already well captures the position information and can learn global features of a sequence of data with long-term dependencies.

Therefore, the output feature h_1 of the service description is:

$$h_1 = f_{lstm}(E) \tag{2}$$

2) Service Name Feature Extraction: Because service name is usually composed of only a few words in our dataset, the way to deal with the name features is different from that of the service description. We embed each service name as a whole into a vector through BERT pre-trained language model, which is represented by f_{bert_pool}. After embedding, service name outputs an n-dimensional vector, we use a fully connected neural network f_{full} with activation function $tanh$ for the service name feature extraction. We assume that x_2 is the name of a service. The embedded features of a service name can be defined as:

$$h_2 = f_{full}(f_{bert_pool}(x_2)) \tag{3}$$

3) Feature Fusion: In practice, the contributions of the service description and service name to the final classification task are different. For feature fusion, we propose a new weighted sum-fusion method, where the weights are learned automatically to re-weight different modalities based on their contributions. We fuse the extracted features of service description h_1 and service name h_2 by multiplying h_1 and h_2 with trainable weights w_1 and w_2, respectively. In the proposed architecture, the lengths of these features are the same. We use vector addition with trainable weights to fuse the features. The output of feature fusion is as follows:

$$h = w_1 \cdot h_1 + w_2 \cdot h_2 \tag{4}$$

4) Multi-head Classification Module: Here, the proposed multi-head classification module is distinguished from conventional classification layers from two main aspects: a) the features are segmented into multiple segments, b) an individual classifier is applied on each part and the classifiers are normalized. In detail, we divide the feature h into M equal non-overlapped feature segments. And then M individual classifiers with different initialization parameters are applied to different feature segments. We normalize the fused features (h) as well as the parameters (w) of each classifier to obtain the final logits (z) for classification. Thus, the calculation of the logits for the final classification results can be defined as:

$$z = \frac{N}{M} \sum_{m=1}^{M} \frac{(w_j^m)^T h^m}{\|w_j^m\| \|h^m\|} \tag{5}$$

where M is the number of heads to equally divide the dimensions of weights and features into M parts and N is the inverse temperature to adjust the smoothness of the probabilities produced by the softmax activation, which will affect the contribution of different categories in gradient calculation. The numerator is a linear classifier without bias and the denominator is the normalization term.

The task of the classifier layer is to obtain the final classification results. In our model, we use softmax activation σ that converts logits z into classification prediction. To train the

Authorized licensed use limited to: Penn State University. Downloaded on June 17,2025 at 14:19:01 UTC from IEEE Xplore. Restrictions apply.

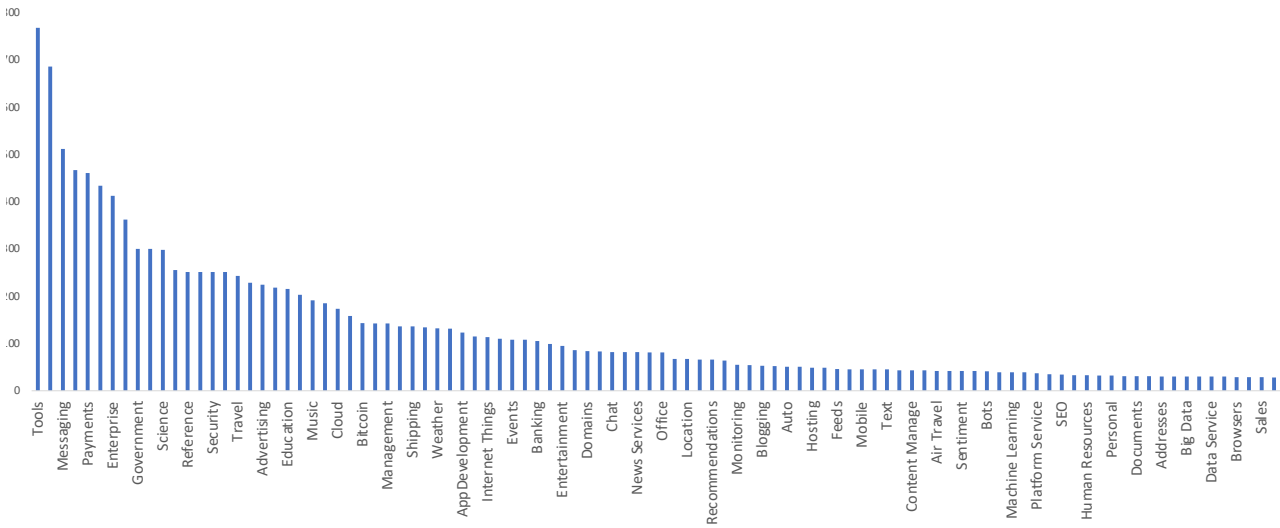

Fig. 2: The visualization of the Service Dataset, which obviously follows a long-tailed distribution.

model, a standard classification loss function $\mathcal{L}_{cls}$ is defined on all the training data pairs $(\boldsymbol{x}, \boldsymbol{y}) \sim \mathcal{D}$ as follows,

$$\mathcal{L}_{cls} = \mathbb{E}_{(\boldsymbol{x},\boldsymbol{y})\sim\mathcal{D}} L_{cross_entropy}(\sigma(z), \boldsymbol{y}) \tag{6}$$

where the standard cross-entropy loss $L_{cross_entropy}$ is used in our method.

The justifications that the proposed method can improve the classification results of the tail categories are two-fold. First, the division of the dimensions of features and classifiers force the model to learn more diverse features, which benefit the learning of the tail categories. Secondly, the feature and classifier normalization further reduce the bias towards the head categories, because the norms of the classifier weights are correlated with the sample numbers of each class. In other words, when joint training the head and tail categories with the standard classifier, the norms of classifier weights of the head categories tend to be larger than that of the tail categories. Thus, if we explicitly normalize the weights and features, the imbalanced effects are reduced.

IV. Evaluation

In this section, we introduce the service dataset, implementation details, evaluation metrics, and the evaluation results of *ServeNet-LT*, which includes the comparison results with other machine learning methods.

A. Service Dataset

1) Service Data Collection: The data of the web services we used are collected from an API sharing platform[1], which is the world's leading source of Internet-based application programming interfaces (APIs). We implement a web crawler through web browser automation tool Selenium[2], the service data are collected and stored in the service dataset *WSDataset* in CSV format. The web services dataset includes all types of services, such as SOAP, REST, RPC, and gRPC. Over 84% of services are REST style. Less than 8% of services are in SOAP style, which makes the additional WSDL hard to be retrieved for service classification. Service specification provided by API sharing platform includes *title*, *description*, *endpoint*, *homepage*, *primary category*, *secondary categories*, *provider*, and *SSL support*.

The dataset contains 15340 service samples and 401 categories. In this paper, we choose service descriptors - *title*, *description*, and *primary category* into account. Under the service classification task, we define *title* as service name, *description* as service description, *primary category* as service classification.

2) Service Dataset Analysis: First, to verify the reliability of the annotation of the data, we randomly sample a small set of data examples as shown in Table I for both head and tail categories. These APIs and the corresponding tags are developed and labeled by expert programmers. Thus, it can be observed that the annotations of the tags are mostly reliable, which supports our development of advanced algorithms for automatic service classification.

Secondly, by further analyzing the data, we observe that the dataset is extremely unbalanced with long-tailed distribution, which is consistent with the fact that most data in the real-world follow a long-tailed distribution. The visualization of the sample distribution of the dataset is shown in Fig. 2. We find that the category with the most samples contains 767 services, while the smallest category contains only one service. Moreover, 41 categories contain 1 number of ser-

[1]http://www.programmableweb.com

[2]https://www.seleniumhq.org

Authorized licensed use limited to: Penn State University. Downloaded on June 17,2025 at 14:19:01 UTC from IEEE Xplore. Restrictions apply.

TABLE I: The example data and the sample annotation details in the dataset

Head Categories			Tail Categories		
Name	ServiceDescription	tag annotation	Name	ServiceDescription	tag annotation
Open Secrets API	The OpenSecrets API provides access to the data displayed on OpenSecrets.org. OpenSecrets performs research to track money in U.S. politics and its effect on elections and public policy. Use the API to access a politician's top contributors, personal financial disclosures for a politician, and more.	Government	MadKudu API	The MadKudu API allows you to lookup predictive informations for a lead and is available to customers on the MadKudu Pro plan. This includes Companies, Persons, Behavior and more. MadKudu automatically analyzes your existing customer data to find patterns that indicate which customers will become your best customers and when is it a right time to engage.	Sales
MySpace API	The MySpace Developer Platform (MDP) allows developers to create applications that interact with MySpace members and their social data. With MDP you will be able to create compelling new products that integrate directly into MySpace pages and get exposure to millions of people around the world.	Social	Hexo API	The Hexo API allows to integrate blogging plugins into applications. As a service, Hexo is a blog framework in Node.js language that allows users to create files in a short amount of time.	Framework
YouTube Content ID API	The YouTube Content ID API allows direct interaction with YouTube's rights management system. Administrators and content owners can obtain metadata, ownership information, and policy information of their content. Additionally, this API can be combined with the YouTube Data API and YouTube Player APIs in order to allow a third party application to upload, and manage videos. The Content ID API requires OAuth 2.0 for authentication.	Video	Hotspot System API	The HotspotSystem.com API allows to implement hotspot features into mobile applications and to bill for internet usage. Protocols available in JSON and XML with SSL, basic HTTP, and Api Key as authentication methods. HotspotSystem provides hotspot management and billing ability to businesses who offer internet to their customers.	Wi-Fi
Viator API	Viator sells tours and activities through their website, eight local-language sites serving European and Japanese markets, and more than 2,000 affiliated sites that include major hotel chains and airlines, online travel agencies, city-specific sites and more. Viator makes it's database of products available as XML. Developers can use the XML to display Viator products on their website as well as in confirmation emails for bookings conducted for other services. Full documentation is not available.	Travel	Tomorrow CO2 Signal API	Tomorrow is an organization that promotes the efficient use of energy resources. The Tomorrow CO2 Signal API provides carbon footprint detection mechanism that makes it possible for a smart device to consume electricity when the carbon footprint is at its lowest. It provides access to information about the source and production procedures of electricity supply as well as the quantity of carbon attributable to its production. The API sends requests and returns responses in JSON format.	Climate
Google Partners API	The Google Partners API is a REST and RPC API. THe Google Partners API gives developers tools to help with advertisers abilities to search certified companies. It also allows for creation of contact leads with the companies. This API is also helpful with audits and monitoring the usage of clients.	Advertising	CoinTent JavaScript API	The CoinTent Javascript API allows developers to integrate a CoinTent widget within websites and applications for CoinTent user logins. CoinTent provides a Chrome plugin to visit websites ad-free and to support websites users like.	Plugins

vices. Similarly, only 41 categories contain more than 100 services. More than half of the categories (the total category is 401 categories), that is, 217 categories contain less than 10 services. More than 70% of the services are concentrated in the top 50 categories and less than 30% in the last 351 categories. In this paper, the service dataset contains 15105 services with 300 categories. we discard the categories with only one service since these categories are not feasible to be divided into training and test splits. First, we choose the top 50 categories to compare with ServerNet and other machine learning methods. And then expand the categories up to 300 categories to further evaluate the effectiveness of the proposed long-tailed classification method.

B. Implementation details

The implementation details of *ServeNet-LT* contain data pre-processing, the network configuration, and training setting.

1) Data Pre-processing: We follow the same strategy in ServeNet [13] for data pre-processing. We divide the dataset into training set and test set in proportion, where 80% of the data are training data and 20% are the test data. For a few categories with only two service samples. we randomly select one service as the training data and the other as the test data. Due to the different lengths of the service descriptions, we sort all the service samples in terms of the length of the service descriptions before feature embedding. Under each

Authorized licensed use limited to: Penn State University. Downloaded on June 17,2025 at 14:19:01 UTC from IEEE Xplore. Restrictions apply.

mini-batch, we apply dynamic padding according to the length of the service descriptions. Two potential benefits can be realized by the dynamic padding. First, the model can pad any length of service description, which is in line with real-world applications. Secondly, it can improve the efficiency of the model and avoid long service descriptions and short service descriptions to be padded in the same mini-batch.

2) Network hyper-parameters: We choose the pre-trained BERT model (*bert_en_uncased_L-12_H-768_A-12*) from Tensorflow Hub[3] to transforms each word of service description and service name into a 768-dimension vector. *ServeNet-LT* contains one bidirectional LSTM layer. The hidden state of LSTM is a 1024-dimension vector.

3) Training hyper-parameters: The total epoch number is 40 and the batch size is 128. The hidden state of LSTM and all biases are initialized to zero. In the multi-head classification layer, we have two important hyper-parameters M and N. Here, we set $N = 16$ and $M = 2$, such that the 1024 dimensional features are segmented into two groups, with 512-dimension features in each group. The classifier weights are initialized by normal initializer with the mean 0 and the standard deviation 0.01, and different heads of the classifier are initialized differently. To avoid over-fitting, we add a dropout layer in service name feature extraction layers of *ServeNet-LT* with drop probability (rate) 0.1.

C. Evaluation Metrics

Top-N accuracy is used to evaluate a classification model. Top-1 accuracy and Top-5 accuracy are commonly used. Top-1 accuracy is a more rigorous evaluation method than Top-5 accuracy, and is also more practical. Top-1 accuracy on the whole dataset is :

$$Accuracy_{top1} = \frac{Num_{top1_correct}}{Num_c} \tag{7}$$

where $Num_{top1_correct}$ is the number of samples with a correct prediction, Num_c is the number of the service samples in the whole dataset. Top-5 accuracy is similar to Top-1 accuracy, except the correct prediction means the Top-5 prediction is correct when Top-5 predicted labels contain the target ground-truth category. In this paper, we use the same evaluation protocol to ensure a fair comparison with the previous method on web service classification [13].

D. Experimental Results and Discussion

The experiments consist of two parts: a) We use the same dataset for a fair comparison with ServeNet [13] and 11 other machine learning methods in 50 categories, including Naive-Bayes, Random Forest (RF) [47], SVM (LDA-Linear-SVM and LDA-RBF-SVM), AdaBoost, CNN, Recurrent-CNN, LSTM, BI-LSTM, C-LSTM. All the conventional models are implemented by scikit-learn library[4]. Moreover, we compare our method with a state-of-the-art method dealing with the data imbalance, which is similar to the long-tailed distributed data named Focal Loss [38]. Focal Loss is a type of loss function designed for imbalanced classification problems by reducing (increasing) the loss values of the simple (hard) samples in the head (tail) categories. It can be seen as a re-weighted cross-entropy loss, where the weights are dynamically assigned based on the predictions. In addition, we also compare with LMA [43], which also aims at solving the long-tailed problem in web services classification tasks. LMA [43] method uses a multi-head attention module, which is inspired by Transformer [42]. b) For the same dataset, we expand the number of categories from 50 categories to 300 categories. We compare our method with the previous ServeNet [13] by gradually expanding the number of categories evenly in the interval [50, 300], with the expanding step size of 50 categories.

[3]https://tfhub.dev/tensorflow

[4]https://scikit-learn.org

TABLE II: Comparison of the proposed method and previous machine learning-based service classification methods

Model	Top-5 Accuracy	Top-1 Accuracy
CNN	58.46	27.60
AdaBoost	64.92	34.93
LDA-Linear-SVM	71.91	33.28
LDA-RBF-SVM	73.84	39.79
Naive-Bayes	78.94	47.74
LSTM	80.10	51.18
RF	80.25	54.29
LMA	83.03	59.67
Recurrent-CNN	84.29	60.02
C-LSTM	84.32	59.24
BI-LSTM	86.70	60.45
ServeNet (Orignal)	88.40	63.31
FocalLoss	90.46	65.80
ServeNet	91.58	69.95
ServeNet-LT	**92.77**	**71.49**

1) Experimental Results of 50 Categories : The experimental results are shown in Table II. It can be observed that the overall trend of the Top-1 accuracy is similar to that of the Top-5 accuracy. The results of ServeNet [13], no matter Top-1 accuracy or Top-5 accuracy, are obviously better than the other 10 machine learning methods. Both the Top-1 accuracy (71.31) and the Top-5 accuracy (92.77) of our new method are higher than ServeNet [13] by 1.54 and 1.19, respectively. By comparing the Focal Loss [38] and the LMA [43] method, as shown in Table II, our new method is 5.69 and 11.82 higher than the Focal Loss [38] and the LMA [43] method in Top-1 accuracy, respectively, because in 50 categories, the data are already long-tailed distributed. Thus, compared with ServeNet [13], our method can effectively deal with the issue of long-tailed data. By dividing the features into individual feature groups and using different classifiers for each group of features, our method learns more diverse and representative features.

2) Experimental Results of the Expanded Categories: We expand the number of categories of web services classification tasks from 50 categories to up to 300 categories. Moreover, with the expansion of the data, the issue of long-tailed data distribution becomes more serious. This also directly leads to significant performance degradation of the tail categories. At

Authorized licensed use limited to: Penn State University. Downloaded on June 17,2025 at 14:19:01 UTC from IEEE Xplore. Restrictions apply.

the same time, because the average number of samples of each category after the expansion is significantly reduced, leading to the further difficulty of discriminative feature learning. Even in this case, our new method still has obvious improvements compared to the original ServeNet [13], Focal Loss [38], and LMA [43] method, as shown in Table III and Table IV.

TABLE III: Top-1 accuracy of the extended categories

Categories	**ServeNet-LT**	ServeNet	FocalLoss	LMA
50 Categories	**71.49**	69.95	65.80	59.67
100 Categories	**65.64**	61.95	60.15	50.74
150 Categories	**63.80**	59.76	57.98	48.22
200 Categories	**62.77**	57.57	56.07	46.11
250 Categories	**60.95**	56.33	51.22	45.84
300 Categories	**59.98**	55.15	51.97	45.50

TABLE IV: Top-5 accuracy of the extended categories

Categories	**ServeNet-LT**	ServeNet	FocalLoss	LMA
50 Categories	**92.77**	91.58	90.46	83.03
100 Categories	**88.39**	85.76	85.80	74.20
150 Categories	**86.59**	83.32	83.74	71.52
200 Categories	**85.29**	81.28	81.55	69.28
250 Categories	**84.00**	78.58	76.05	67.94
300 Categories	**82.91**	77.73	75.54	67.32

Overall, it can be observed that the proposed *ServeNet-LT* outperforms the original ServeNet [13] as well as the Focal Loss [38] and LMA [43] method designed for the long-tailed classification on all the experiments. The Top-1 accuracy and the Top-5 accuracy have similar trends among different methods in Table III and Table IV. In addition, Top-1 accuracy is the most effective and intuitive evaluation protocol in practice. Thus, we analyze all the results based on the Top-1 accuracy here.

On one hand, with the increasing of the number of categories, all the methods observe an overall trend of performance degradation. For example, In Table III, the *ServeNet-LT*, ServeNet [13], Focal Loss [38] and LMA [43] obtain the accuracy of 71.49%, 69.95%, 65.80%, and 59.67%. When the number of categories grows to 300, the accuracy of the three methods drops to 59.98%, 55.15%, 51.97%, and 45.50%. This can be explained that when more categories are involved, the inter-class variance and the average number of samples per category both become smaller, which increases the difficulty of the learning of discriminative features for all the classification methods.

On the other hand, the performance gains of our proposed method compared to the baseline methods gradually increase as more categories are involved. In detail, compared with the original ServeNet [13], our method improves 1.54% accuracy on the 50-category data. When 300 categories are involved, the proposed method outperforms ServeNet by 4.83%. This is because when more categories are included, the long-tailed phenomenon of data is more obvious. We propose a multi-head classifier, which segments the features into different parts that capture different aspects of the data to learn more diverse features that may be ignored by a single classifier. These features are supposed to be beneficial for the tail classes. Moreover, we normalize both the segments of features and the classifier weights, such that the classifier is less biased towards the head categories. Our method explicitly deals with the long-tailed issue and the experiments further verify the effectiveness of our method on long-tailed classification of web services.

3) Analyses of the Head and Tail Categories: We also analyse the head and tail categories separately to show the effectiveness of our method that can improve the performance of the tail categories while not degenerating the head categories.

In this paper, we rank all the categories in descending order based on the sample numbers and define the categories with large sample numbers categories that account for 80% of the total samples as the *head categories* and the rest categories that contain 20% of the total samples as the *tail categories*. Although our method has improved in both the head category and the tail category, the improvement of the tail category is more obvious. The Table V shows more detailed experimental results. Under the tail categories in the 100-category task, *ServeNet-LT* achieves 50.63% Top-1 accuracy, which is 5.48% higher than that of the ServeNet [13](45.15%). As more categories are involved, the sample numbers in the tail categories increase much slower than the head categories. For example, when the categories are expanded from 200 to 250, the number of samples in the tail categories only increases by 12, while the number of samples in the head categories increases by 84. Even in such extremely imbalanced data, our method still achieves the improvement of the tail category. As for the performance of the tail categories in the 300-category task, the Top-1 accuracy of *ServeNet-LT* is still 3.19% higher than that of the ServeNet [13]. The results show that our method is very effective on long-tailed data.

4) Ablation Study: To further analyze the effectiveness of each part of the proposed normalized multi-head classification module in the proposed *ServeNet-LT* method, we conduct two ablation studies: a) Evaluation of the effect of feature and classifier weights normalization, and b) Evaluation of the influence of the number of heads.

Evaluation of feature and classifier weights normalization: As reported in Table VI, the *ServeNet-LT* achieves the best performance under the feature and classifier weights normalization. The *ServeNet-LT* improves all categories with respect to both the Top-1 and Top-5 accuracy, because normalization reduces the bias towards the head categories.

Evaluation of the influence of the number of heads: To further study the multi-head strategy on the *ServeNet-LT*, we evaluate the hyperparameter M by setting $M = 2$ and $M = 4$ in Table VII. It shows that the advantage of the proposed *ServeNet-LT* model benefits from the feature splitting. Moreover, $M = 2$ works best for all the datasets.

5) Limitation: While expanding the categories, *ServeNet-LT* effectively alleviates the impact of the long-tailed distribution of data. However, another key reason that affects the

Authorized licensed use limited to: Penn State University. Downloaded on June 17,2025 at 14:19:01 UTC from IEEE Xplore. Restrictions apply.

TABLE V: Comparison of Top-1 accuracy on tail and head categories

Top-1 accuracy of tail categories			Top-1 accuracy of head categories		
Tail Category	**ServeNet-LT**	ServeNet	Head Category	**ServeNet-LT**	ServeNet
50 Categories	**65.57**	61.23	50 Categories	**73.03**	72.20
100 Categories	**50.63**	45.15	100 Categories	**69.44**	65.27
150 Categories	**48.52**	43.84	150 Categories	**67.50**	63.58
200 Categories	**43.72**	38.44	200 Categories	**67.14**	62.29
250 Categories	**38.96**	36.32	250 Categories	**65.50**	59.56
300 Categories	**34.71**	31.52	300 Categories	**65.34**	60.21

TABLE VI: Ablation study of the feature and classifier weights normalization in *ServeNet-LT* on Top-1 and Top-5 accuracy

Top-1 accuracy				Top-5 accuracy			
Categories	**ServeNet-LT Normalization**	No-Normalization	ServeNet	Categories	**ServeNet-LT Normalization**	No-Normalization	ServeNet
50 Categories	**71.49**	70.59	69.95	50 Categories	**92.77**	91.96	91.58
100 Categories	**65.64**	63.87	61.95	100 Categories	**88.39**	87.41	85.76
200 Categories	**62.77**	60.24	57.57	200 Categories	**85.29**	83.28	81.28
300 Categories	**59.98**	57.79	55.15	300 Categories	**82.91**	80.56	77.73

TABLE VII: Evaluation results of the number of heads in *ServeNet-LT* on Top-1 accuracy and Top-5 accuracy

Top-1 accuracy				Top-5 accuracy			
Categories	**ServeNet-LT 2-Head**	ServeNet-LT 4-Head	ServeNet	Categories	**ServeNet-LT 2-Head**	ServeNet-LT 4-Head	ServeNet
50 Categories	**71.49**	70.82	69.95	50 Categories	**92.77**	92.18	91.58
100 Categories	**65.64**	65.42	61.95	100 Categories	**88.39**	88.32	85.76
200 Categories	**62.77**	62.00	57.57	200 Categories	**85.29**	84.48	81.28
300 Categories	**59.98**	59.41	55.15	300 Categories	**82.91**	81.96	77.73

accuracy of web services classification is the dataset. The most straightforward way to deal with this problem is to collect more high-quality data, especially for the tail categories. One way is to expand the current dataset with respect to the number of samples. Another potential solution is to introduce external knowledge mapping to further expand the trainable data.

V. Conclusion and Future Work

This paper introduces a novel deep neural network, which expands the task of web services classification from 50 categories to 300 categories, and effectively alleviates the impact of long-tailed distribution of data. Extensive experiments are conducted on the web services datasets with up to 300 categories. The experimental results and in-depth analyses verify that the proposed *ServeNet-LT* can effectively deal with the long-tailed distribution of data and achieves the state-of-the-art results on all the settings of the web services datasets. The source code and the dataset of this paper will be released once the paper is accepted.

In the future, we plan to introduce more data sources as the model input, such as service URL, service signature, and provider, etc, which provides more complementary information to improve the performance of service classification. For the model output, we hope to achieve higher accuracy and more information. For example, some services may belong to different categories at the same time, and we wish to precisely identify all categories of the services. For the model, we hope to further improve the algorithm via more advanced approaches such as graph neural network and transfer learning. In addition, automatic service discovery classification [48] [49] is also meaningful to do, like PM2PT [50] we proposed before. We always believe that our work can promote more research in service computing and other related fields of software engineering.

VI. Acknowledgment

This work was supported by the National Natural Science Foundation of China under Grant No. 62006012 and No. 61732019, and the Opening Project of Shanghai Trusted Industrial Control Platform No. KH54327801.

References

[1] Y. Yang, Q. Zu, P. Liu, D. Ouyang, and X. Li, "MicroShare: Privacy-preserved medical resource sharing through microservice architecture," *International journal of biological sciences*, vol. 14, no. 8, p. 907, 2018.

[2] A. Mintchev, "Interoperability among service registry implementations: Is UDDI standard enough?" in *2008 IEEE International Conference on Web Services (ICWS'08)*, Sep. 2008, pp. 724–731.

[3] J. E. Hajlaoui, M. N. Omri, D. Benslimane, and M. Barhamgi, "QoS based framework for configurable IaaS cloud services discovery," in *2017 IEEE International Conference on Web Services (ICWS'17)*, Jun. 2017, pp. 460–467.

[4] P. Hasselmeyer, "On service discovery process types," in *Proceedings of the 3th International Conference Service-Oriented Computing (ICSOC'05)*, Dec. 2005, pp. 144–156.

[5] B. Cheng, S. Zhao, C. Li, and J. Chen, "MISDA: web services discovery approach based on mining interface semantics," in *IEEE International Conference on Web Services (ICWS'16)*, June 2016, pp. 332–339.

[6] H. Han, W. Y. Wang, and B. H. Mao, *Borderline-SMOTE: A New Over-Sampling Method in Imbalanced Data Sets Learning*. Springer Berlin Heidelberg, 2005.

[7] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, no. 1, pp. 321–357, 2002.

Authorized licensed use limited to: Penn State University. Downloaded on June 17,2025 at 14:19:01 UTC from IEEE Xplore. Restrictions apply.

[8] C. Drummond, R. C. Holte *et al.*, "C4. 5, class imbalance, and cost sensitivity: why under-sampling beats over-sampling," in *Workshop on learning from imbalanced datasets II*, vol. 11. Citeseer, 2003, pp. 1–8.

[9] M. Ren, W. Zeng, B. Yang, and R. Urtasun, "Learning to reweight examples for robust deep learning," in *International Conference on Machine Learning*. PMLR, 2018, pp. 4334–4343.

[10] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, "Class-balanced loss based on effective number of samples," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 9268–9277.

[11] S. H. Khan, M. Hayat, M. Bennamoun, F. A. Sohel, and R. Togneri, "Cost-sensitive learning of deep feature representations from imbalanced data," *IEEE transactions on neural networks and learning systems*, vol. 29, no. 8, pp. 3573–3587, 2017.

[12] J. Tan, C. Wang, B. Li, Q. Li, W. Ouyang, C. Yin, and J. Yan, "Equalization loss for long-tailed object recognition," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 11 662–11 671.

[13] Y. Yang, N. Qamar, P. Liu, K. Grolinger, and Z. Liao, "ServeNet: A deep neural network for web services classification," in *2020 IEEE International Conference on Web Services (ICWS)*, 2020.

[14] L. Fang, L. Wang, M. Li, J. Zhao, Y. Zou, and L. Shao, "Towards automatic tagging for web services," in *2012 IEEE 19th International Conference on Web Services*, Jun. 2012, pp. 528–535.

[15] I. Katakis, G. Meditskos, G. Tsoumakas, N. Bassiliades, and Vlahavas, "On the combination of textual and semantic descriptions for automated semantic web service classification," in *Proceedings of IFIP International Conference on Artificial Intelligence Applications and Innovations (AIAI'09)*, Apr 2009, pp. 95–104.

[16] H. Wang, Y. Shi, X. Zhou, Q. Zhou, S. Shao, and A. Bouguettaya, "Web service classification using support vector machine," in *Proceedings of IEEE International Conference on Tools with Artificial Intelligence (ICTAI'10)*, Oct. 2010, pp. 3–6.

[17] J. Su, J. S. Shirab, and S. Matwin, "Large scale text classification using semisupervised multinomial naive bayes," in *Proceedings of the 28th International Conference on Machine Learning, (ICML'11)*, Jun. 2011, pp. 97–104.

[18] P. Zhang, S. Shu, and M. Zhou, "An online fault detection model and strategies based on svm-grid in clouds," *IEEE/CAA Journal of Automatica Sinica*, vol. 5, no. 2, pp. 445–456, Mar 2018.

[19] A. M. Almalawi, A. Fahad, Z. Tari, M. A. Cheema, and I. Khalil, "*k*nnvwc: An efficient k-nearest neighbors approach based on various-widths clustering," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 1, pp. 68–81, 2016.

[20] S. Maliah and G. Shani, "MDP-Based cost sensitive classification using decision trees," in *Proceedings of the Thirty-Second Conference on Artificial Intelligence (AAAI'18)*, Feb. 2018.

[21] G. M. Kapitsaki, "Annotating web service sections with combined classification," in *2014 IEEE International Conference on Web Services*, 2014, pp. 622–629.

[22] Y. Cong, B. Chen, H. Liu, and M. Zhou, "Deep latent dirichlet allocation with topic-layer-adaptive stochastic gradient riemannian MCMC," in *Proceedings of the 34th International Conference on Machine Learning (ICML'17)*, 2017, pp. 864–873.

[23] X. Liu, S. Agarwal, C. Ding, and Q. Yu, "An LDA-SVM active learning framework for web service classification," in *Proceedings of IEEE International Conference on Web Services (ICWS'16)*, Jun. 2016, pp. 49–56.

[24] T. Liang, L. Chen, J. Wu, and A. Bouguettaya, "Exploiting heterogeneous information for tag recommendation in API management," in *Proceedings of IEEE International Conference on Web Services (ICWS'16)*, Jun. 2016, pp. 436–443.

[25] W. Shi, X. Liu, and Q. Yu, "Correlation-aware multi-label active learning for web service tag recommendation," in *Proceedings of IEEE International Conference on Web Services (ICWS'17)*, Jun. 2017, pp. 229–236.

[26] Y. Bengio, A. Courville, and P. Vincent, "Representation learning: A review and new perspectives," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, pp. 1798–1828, Aug 2013.

[27] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *nature*, vol. 521, no. 7553, p. 436, 2015.

[28] H. Bagherinezhad, M. Rastegari, and A. Farhadi, "LCNN: lookup-based convolutional neural network," in *2017 IEEE Conference on Computer Vision and Pattern Recognition, (CVPR'17)*, Jul. 2017, pp. 860–869.

[29] J. Deng, W. Dong, R. Socher, L. Li, K. Li, and F. Li, "ImageNet: A large-scale hierarchical image database," in *2009 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'09)*, Jun. 2009, pp. 248–255.

[30] R. Johnson and T. Zhang, "Supervised and semi-supervised text categorization using lstm for region embeddings," in *Proceedings of the 33rd International Conference on Machine Learning (ICML'16)*, Jun. 2016, pp. 526–534.

[31] S. Wang, M. Huang, and Z. Deng, "Densely connected CNN with multi-scale feature attention for text classification," in *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence, (IJCAI'18)*, Jul. 2018, pp. 4468–4474.

[32] Y. Yang, W. Ke, W. Wang, and Y. Zhao, "Deep learning for web services classification," in *2019 IEEE International Conference on Web Services (ICWS'19)*, July 2019, pp. 440–442.

[33] Yang, Jing, Yilong, Li, Xiaoshan, Wang, and Weiru, "An integrated framework for semantic service composition using answer set programming," *International Journal of Web Services Research*, 2014.

[34] G. Zou, Z. Qin, Q. He, P. Wang, B. Zhang, and Y. Gan, "DeepWSC: A novel framework with deep neural network for web service clustering," in *2019 IEEE International Conference on Web Services (ICWS)*, 2019, pp. 434–436.

[35] L. A. Bing, L. A. Zhi, and B. Yy, "Residual attention graph convolutional network for web services classification," *Neurocomputing*, 2021.

[36] X. Wang, J. Liu, X. Liu, X. Cui, and H. Wu, "A novel dual-graph convolutional network based web service classification framework," in *2020 IEEE International Conference on Web Services (ICWS)*, 2020.

[37] S. Li, H. Luo, and G. Zhao, "bi-hptm: An effective semantic match-making model for web service discovery," in *2020 IEEE International Conference on Web Services (ICWS)*. IEEE, 2020, pp. 433–440.

[38] T. Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PP, no. 99, pp. 2999–3007, 2017.

[39] B. Kang, S. Xie, M. Rohrbach, Z. Yan, A. Gordo, J. Feng, and Y. Kalantidis, "Decoupling representation and classifier for long-tailed recognition," 2019.

[40] K. Tang, J. Huang, and H. Zhang, "Long-tailed classification by keeping the good and removing the bad momentum causal effect," in *NeurIPS*, 2020.

[41] Y. Wang, W. Gan, J. Yang, W. Wu, and J. Yan, "Dynamic curriculum learning for imbalanced data classification," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 5017–5026.

[42] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *arXiv*, 2017.

[43] G. Zou, H. Wu, S. Yang, M. Jiang, and Y. Gan, "Lma: label-based multi-head attentive model for long-tail web service classification," *International Journal of Computational Science and Engineering*, vol. 23, no. 2, p. 158, 2020.

[44] Z. Liu, Z. Miao, X. Zhan, J. Wang, and S. X. Yu, "Large-scale long-tailed recognition in an open world," in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[45] M. A. Jamal, M. Brown, M.-H. Yang, L. Wang, and B. Gong, "Rethinking class-balanced methods for long-tailed visual recognition from a domain adaptation perspective," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 7610–7619.

[46] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on knowledge and data engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.

[47] S. Haghiri, D. Garreau, and U. von Luxburg, "Comparison-based random forests," in *Proceedings of the 35th International Conference on Machine Learning (ICML'18)*, Jul. 2018, pp. 1871–1880.

[48] Y. Yang, X. Li, W. Ke, and Z. Liu, "Automated prototype generation from formal requirements model," *IEEE Transactions on Reliability*, vol. 69, no. 2, pp. 632–656, 2020.

[49] Y. Yang, W. Ke, and X. Li, "RM2PT: Requirements validation through automatic prototyping," in *Proceedings of 27th International Requirements Engineering Conference (RE'19)*, Sep. 2019, pp. 484–485.

[50] Y. Yang, X. Li, Z. Liu, and W. Ke, "RM2PT: A tool for automated prototype generation from requirements model," in *Proceedings of the 41th International Conference on Software Engineering: Companion Proceedings (ICSE'19)*, May. 2019, pp. 59–62.

Authorized licensed use limited to: Penn State University. Downloaded on June 17,2025 at 14:19:01 UTC from IEEE Xplore. Restrictions apply.